Credit Card Fraud Detection Pipeline

1. Executive Summary & Problem Formulation

Core Challenges Identified

- **Severe Class Imbalance:** The positive target class (is_fraud = 1) constitutes only ~1.51% of the dataset (121 positive cases out of 8,000)... Standard accuracy metrics would yield a deceptive 98.49% baseline simply by predicting non fraud for all samples…
- **Metric Selection:** F1-score serves as the primary evaluation metric, balancing the cost of missed fraudulent transactions (False Negatives) against customer friction from flagged legitimate purchases (False Positives)...

2. Feature Engineering & Preprocessing

Tree based models require numeric matrices. The categorical variable merchant_category (spanning categories such as Grocery, Food, Travel, Clothing, Electronics) was transformed via One Hot Encoding.

Raw Categorical Feature	One Hot Encoded Feature Columns	Sample Value
merchant_category = "Grocery"	merchant_category_Grocery	1
merchant_category = "Grocery"	merchant_category_Clothing	0
merchant_category = "Grocery"	merchant_category_Electronics	0

- **Why One Hot Encoding?** Unlike Label Encoding which assigns sequential integers (0, 1, 2, ...) and accidentally imposes an artificial spatial order on unordered categories (e.g., implying Travel > Food), One Hot Encoding converts each category into an independent indicator feature, ensuring tree splits make no false ordinal assumptions…
- **Feature Alignment:** Test set features were explicitly aligned using reindex(columns=FEATURE_NAMES, fill_value=0) to guarantee exact matrix compatibility between training and inference environments…

3. Algorithm Selection & Mathematical Rationale

Gradient Boosted Decision Trees (GBDT) consistently outperform Deep Neural Networks on tabular data of this scale due to their inductive bias toward axis-aligned decision boundaries...We selected **LightGBM.**

Why LightGBM?

- **Leaf Wise (Best First) Tree Growth:** Standard GBDTs split trees level wise. LightGBM splits the leaf with the maximum delta loss achieving lower loss in fewer iterations…

- **Class Imbalance Handling:** The algorithm uses a scale parameter calculated directly from the class ratio:
  *scale_pos_weight = (N_negative) / (N_positive) = 7879 / 121 ≈ 65.12*This modifies gradient computations, scaling loss gradients for minority-class errors by 65.12x and pushing decision boundaries deeper into positive-instance regions.

4. Hyperparameter Optimization & Validation Strategy

To prevent data leakage during hyperparameter selection, a **5 Fold Stratified Cross Validation** scheme was implemented preserving the ~1.51% positive target ratio across all training and validation splits…

Optimal Configuration

- n_estimators: 150 Prevents over fitting on small datasets while providing sufficient capacity...
- learning_rate: 0.10 Standard shrinkage step size for steady gradient descent convergence...
- num_leaves: 31 Controls model complexity keeps tree capacity bounded…
- min_child_samples: 5 preventing the model from memorizing individual outlier fraud points…

Decision Threshold Search

The default decision threshold of 0.50 is suboptimal for imbalanced data. By sweeping predicted probabilities over $t \in [0.005, 0.990]$, an optimal threshold was identified…
Best F1-Score = 0.9917 @ t = 0.100

5. Model Benchmarks & Comparison

Model Architecture	Stratified 5-Fold CV AUC	Best OOF F1-Score	Optimal Threshold (t)
Baseline LightGBM (Default Params)	0.9999	0.9874	0.018
Tuned LightGBM (Selected Winner)	**0.9999**	**0.9917**	**0.100**
Average Ensemble (LGBM + XGB + CatBoost)	0.9999	0.9917	0.562

Key Observation: Stacking models with a Logistic Regression meta learner underperformed (F1 = 0.9750)... With only 121 positive examples partitioning training data further for meta learner training introduces variance loss that outweighs stacking benefits… Single tuned LightGBM remains the leanest and most effective architecture…